Keyword Search over Tables in Data Catalogs

Ph.D. Qualifying Exam
Minh Phan
University of Wisconsin-Madison

Committee:
Rahul Chatterjee, AnHai Doan, Rishab Goyal

Motivation

- **Organizations increasingly have many datasets**
  - Relational databases, noSQL databases, tables, files, images, emails, documents, …
- **Most projects use only a few datasets**
- **But finding them in a sea of datasets is often very difficult**

- **To solve this, organizations use data catalogs**

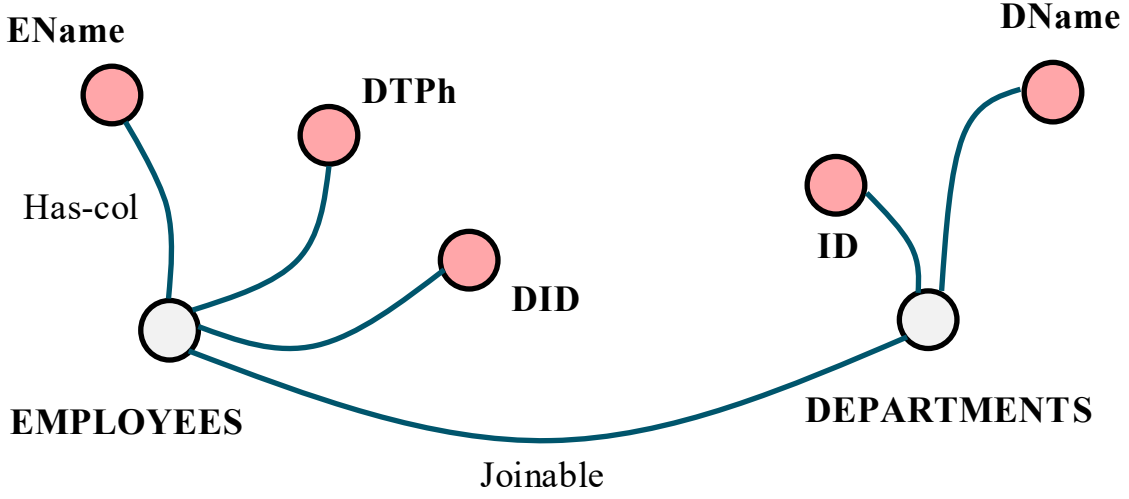
Example 1: Companies

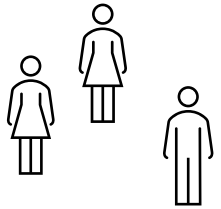
EMPLOYEES

EName	DTPh	DID
Dave Smith	4399	d_1
Jane Miller	5603	d_2
Mike Davis	2862	d_1

DEPARTMENTS

ID	DName
d_1	Sales
d_2	Legal

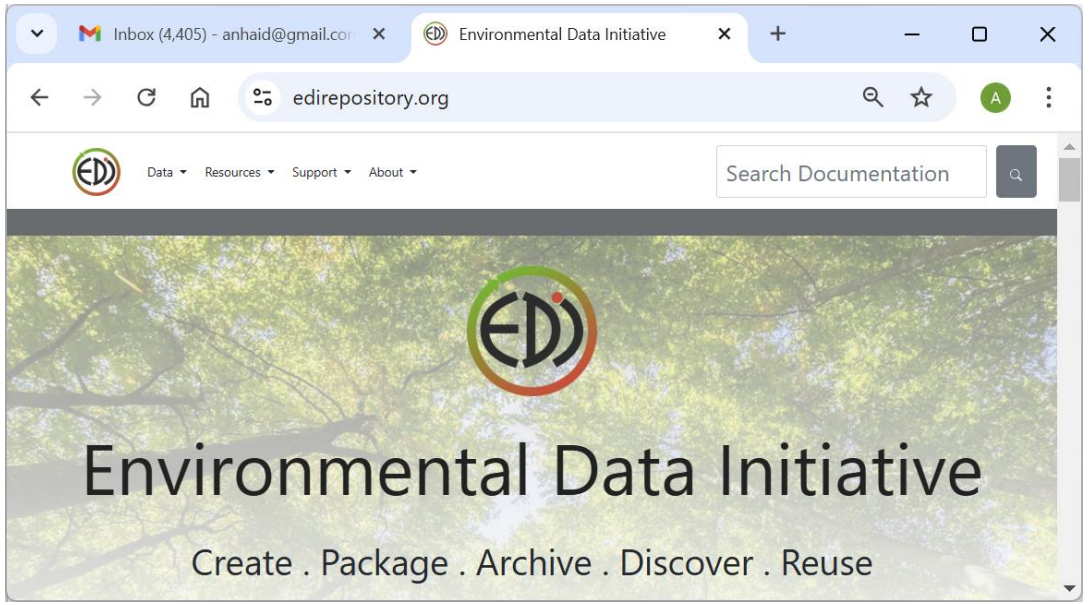


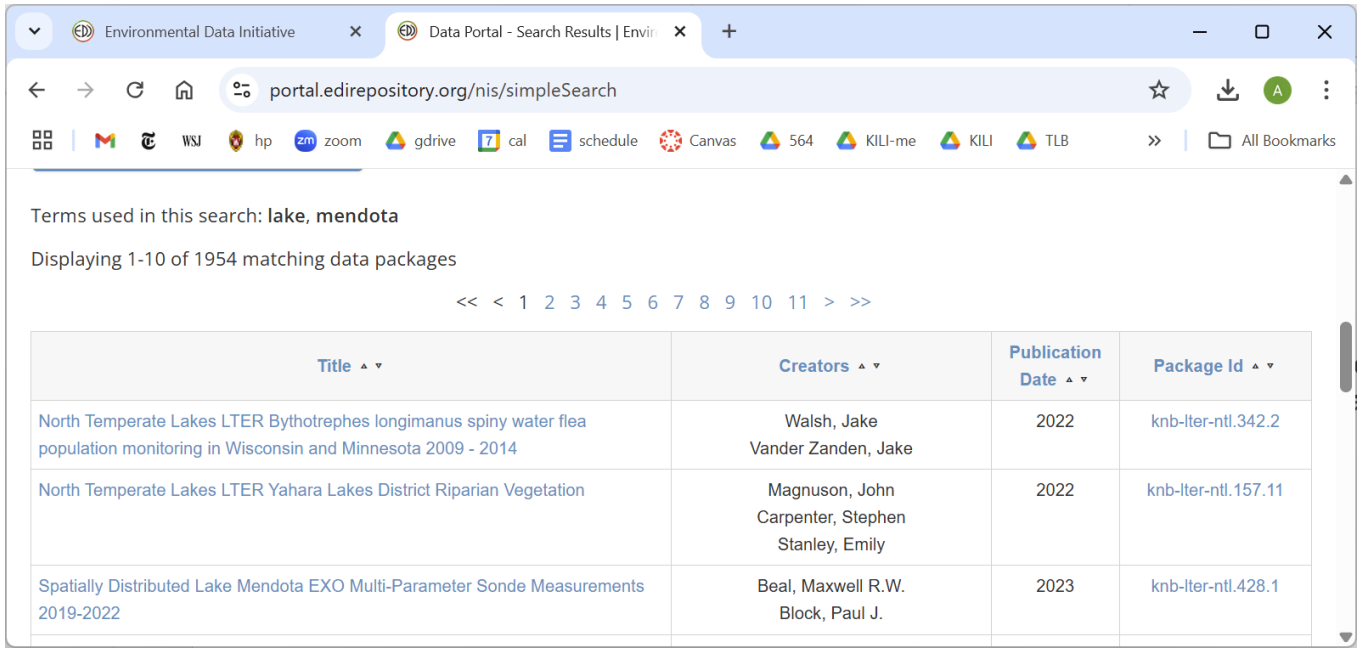


Keyword search
NL querying
Browsing

- **Many catalog vendors**
  - Alation, Amazon, Ataccama, Collibra, Databricks, Data.world, Google, IBM, Informatica, Microsoft, Waterline

Example 2: Domain Sciences



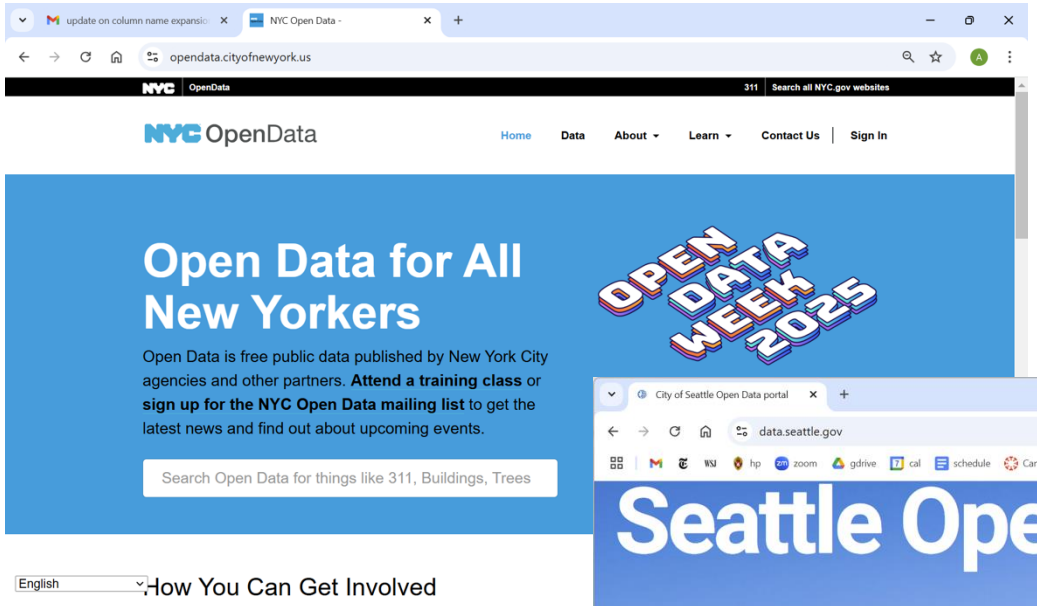


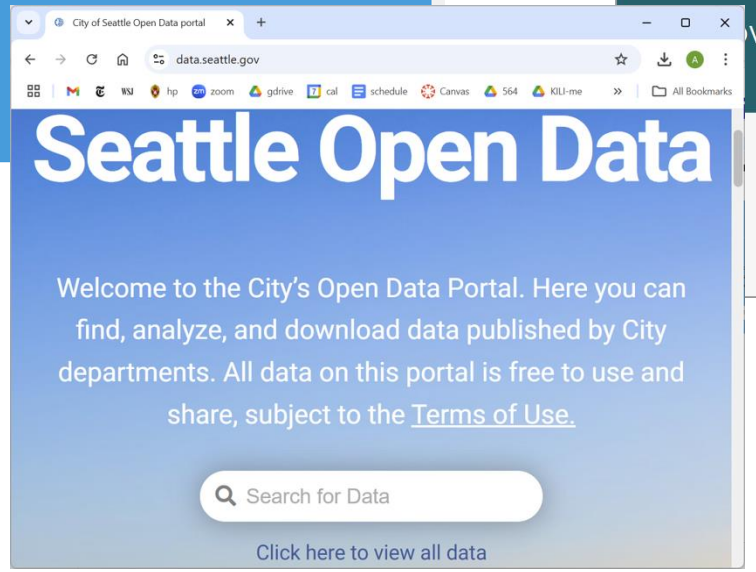
Since 2007
87K data packages, 18K tables
60TB storage, 10K downloads / week
14 employees, $1M burn rate / year

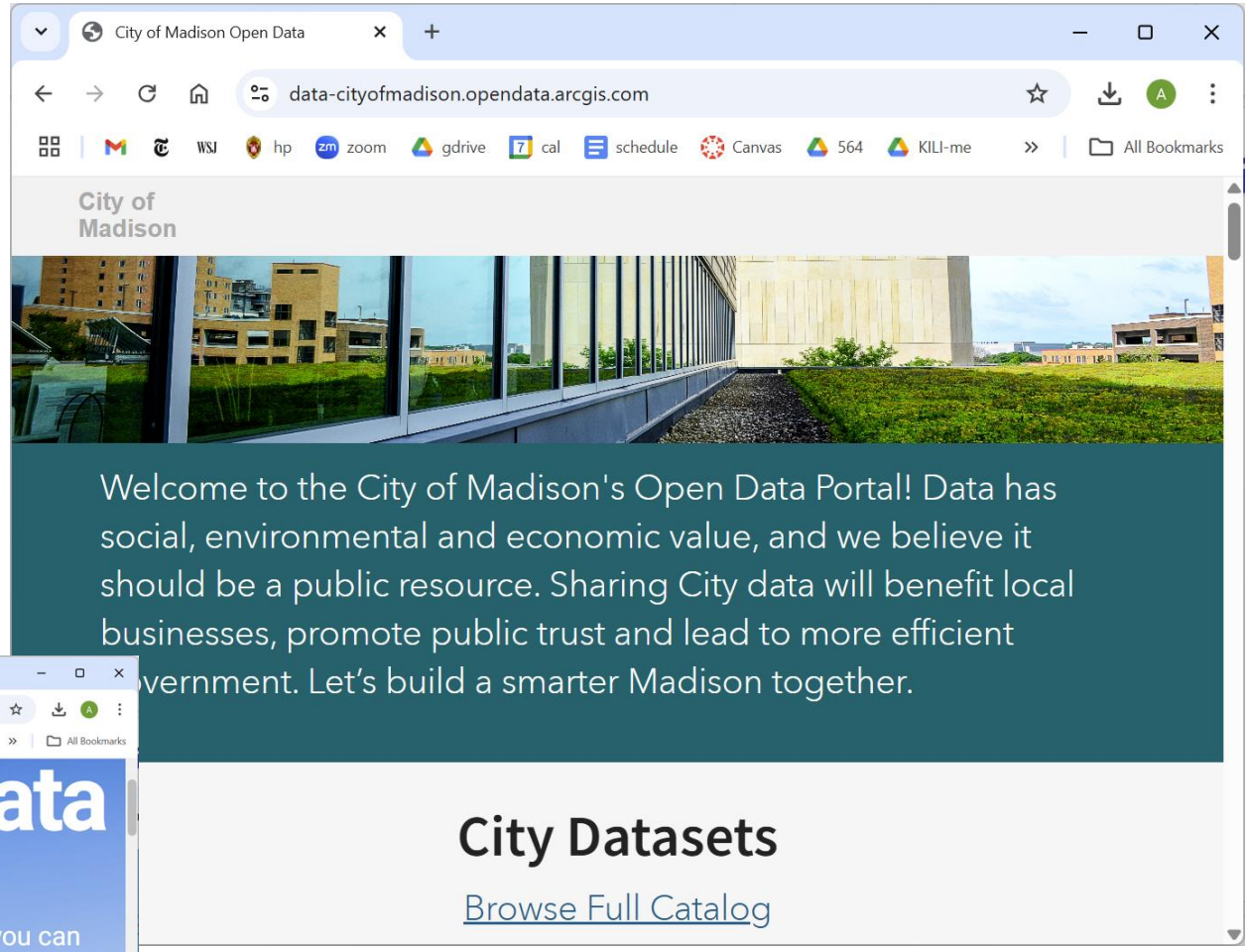
- **Other lakes in environmental science**
  - Dryad, CUAHSI, Zenodo, Figshare, BCO-DMO, Artic Data Center, IDigBio

Example 3: Government Agencies

Data is a valuable asset
Taxpayers pay for it
So they should have access







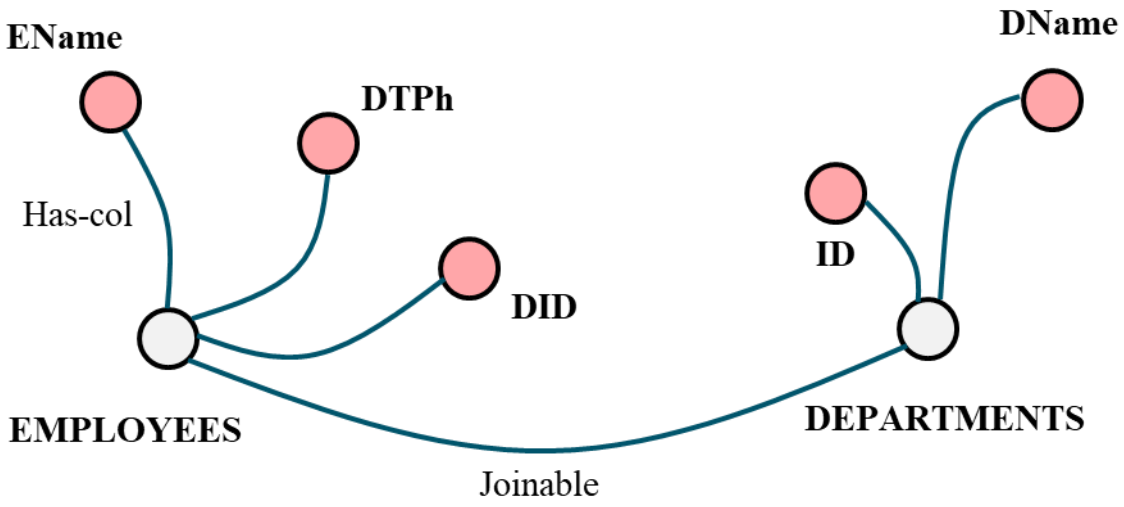
The SmartCat Project @ Wisconsin

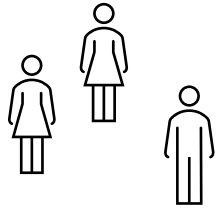
EMPLOYEES

EName	DTPh	DID
Dave Smith	4399	d_1
Jane Miller	5603	d_2
Mike Davis	2862	d_1

DEPARTMENTS

ID	DName
d_1	Sales
d_2	Legal





Keyword search
NL querying
Browsing

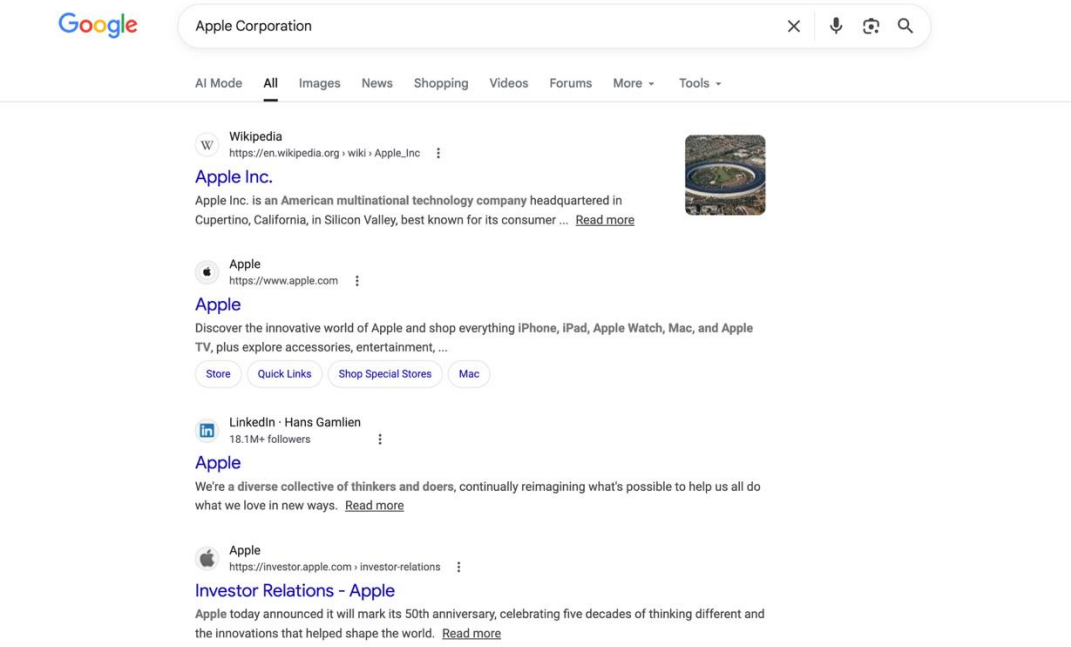
- **Inferring metadata**
- **Ways to find datasets**
- **Curation**
- **Scaling**
- **Handling changes**

This Project: Keyword Search to Find Tables

- **Background**
  - KWS over documents
  - KWS over tables
- **Goals of this project**
- **Collection of datasets**
  - The most comprehensive collection today
- **Tuning KWS systems**
- **Applying KWS systems to tables enriched with new metadata**

KWS over Documents

- **Long-standing problem in Information Science and Web Search**
- **Researched since the 1950's**



Collection of documents

d_1 = “Apple Inc.”
d_2 = “IBM Corporation”
d_3 = “Oracle Corporation”
d_4 = “Tesla Inc.”

q = “Apple Corporation” ⟶

1. d_1
2. d_3
3. d_2

- **In decreasing order of sim(q,d)**

TF/IDF Similarity Score

- **Tokenize query q into a set of tokens**
  - q = “Apple Corporation” => B_q = {apple, corporation}
- **Tokenize each document d into a set of tokens**
  - d = “Apple Inc. Products” => B_d = {apple, inc, product}
- **Let T be the vocabulary = set of all tokens (also called “terms”)**

- **Convert B_q into a feature vector v_q: one feature per token in T**
- **Convert B_d into a feature vector v_d: one feature per token in T**

- **Compute sim(q,d) = cosine(v_q,v_d)**

Example

"Apple Inc." → {"Apple", "Inc."}

- **Tokenize documents**

$x = aab \Rightarrow B_x = \{a, a, b\}$

$y = ac \Rightarrow B_y = \{a, c\}$

$z = a \Rightarrow B_z = \{a\}$

token = term

- **Computer TF and IDF**

tf(a, x) = 2 idf(a) = 3/3 = 1

tf(b, x) = 1 idf(b) = 3/1 = 3

... idf(c) = 3/1 = 3

tf(c, z) = 0

docs

docs containing c

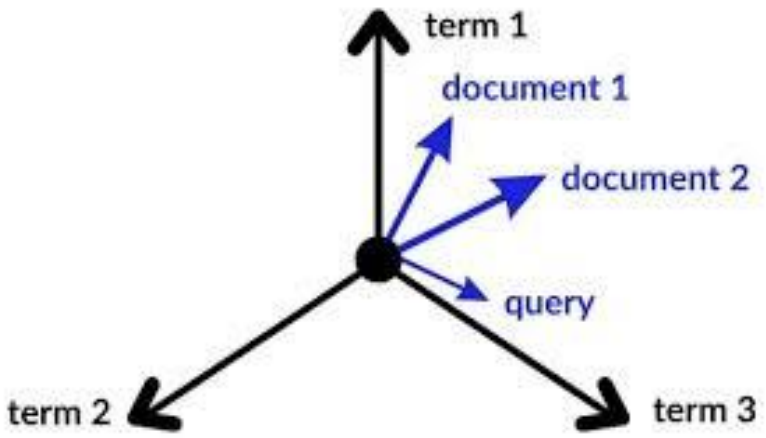
- **Convert each doc into a feature vector**
  - v_d(t) = tf(t,d) * idf(t)

tf(a, x) * idf(a)

	a	b	c
v_x	2	3	0
v_y	3	0	3
v_z	3	0	0

- **Convert query q into a feature vector**
  - Use the same process

Example



- **TF/IDF score of p and q is the cosine of the angle between v_p and v_q**
  - $s(p,q) = \frac{\sum_{\{t \in T\}} v_p(t) * v_q(t)}{\sqrt{\sum_{\{t \in T\}} v_p(t)^2} \sqrt{\sum_{\{t \in T\}} v_q(t)^2}}$

Intuition behind TF/IDF Score

$x = aab \Rightarrow B_x = \{a, a, b\}$

$y = ac \Rightarrow B_y = \{a, c\}$

$z = a \Rightarrow B_z = \{a\}$

$tf(a, x) = 2$ $\quad$ $idf(a) = 3/3 = 1$

$tf(b, x) = 1$ $\quad$ $idf(b) = 3/1 = 3$

... $\quad$ $idf(c) = 3/1 = 3$

$tf(c, z) = 0$

	a	b	c
v_x	2	3	0
v_y	3	0	3
v_z	3	0	0

Collection of documents

d_1 = "Apple Inc."
d_2 = "IBM Corporation"
d_3 = "Oracle Corporation"
d_4 = "Tesla Inc."

q = "Apple Corporation" $\longrightarrow$

1. d_1
2. d_3
3. d_2

Implementing KWS with TF/IDF Scoring

- **Use inverted index**

Collection of documents

d_1 = "Apple Inc."
d_2 = "IBM Corporation"
d_3 = "Oracle Corporation"
d_4 = "Tesla Inc."

q = "Apple Corporation" ⟶

1. d_1
2. d_3
3. d_2

- **In practice, use Lucene & Elastic Search**
  - Use BM25† TF/IDF similarity function

$$BM25(d_x, d_y) = \sum_{t \in d_x} idf(t) \frac{\boldsymbol{tf(t, d_y) \cdot (k_1 + 1)}}{\boldsymbol{tf(t, d_y) + k_1 \cdot (1 - b + \frac{b|d_y|}{l})}}$$

Managing Documents with Multiple Sections

Document d_i

f_1
f_2
$\vdots$
f_n

Query Q

q
q
$\vdots$
q

$$s(q, d_i) = \Sigma_j \; w_j \cdot BM25(q, f_j)$$

- **How to find, aka tune, tokenizers and weights?**

Tuning Tokenizers and Weights

- **Ideally, we have queries with known relevant tables**
  - Optimize parameters for a retrieval metric [Taylor et al., CIKM 2006]
    - Grid search over configs to maximize nDCG@10
- **Sometimes, we only have queries**
  - Rely on weak signals [Joachims, KDD 2002; Roitman, CEUR 2022]
    - Click-through rates, heuristics
- **In practice, labeled data is expensive and often unavailable**

Recent Developments

- **KWS with TF/IDF => sparse retrieval**
- **KWS with embedding vectors => dense retrieval**
- **Many solutions combine both**

- **KWS with embedding vectors is still very hard to do well**
  - Hard to create good index
  - Embedding vectors are large

- **So most real-world systems still use TF/IDF**
  - Our current focus

This Project: Keyword Search to Find Tables

- **Background**
  - KWS over documents
  - KWS over tables
- **Goals of this project**
- **Collection of datasets**
  - The most comprehensive collection today
- **Tuning KWS systems**
- **Applying KWS systems to tables enriched with new metadata**

Keyword Search to Find Tables

Find me all tables that have employee phones
Q = “employee phone”

EMPLOYEES

EName	DTPh	DID
Dave Smith	4399	d_1
Jane Miller	5603	d_2
Mike Davis	2862	d_1

<title> EMPLOYEES</title>
<columns> Ename DTPh DID</columns>
<context>This table describes …</context>

DEPARTMENTS

ID	DName
d_1	Sales
d_2	Legal

<title> DEPARTMENTS</title>
<columns>ID DName</columns>
<context>This table was created …</context>

$$s(q, d) = w_1 \cdot BM25(t_1(q), t_1(\text{tab name})) + w_2 \cdot BM25(t_2(q), t_2(\text{col names})) + w_3 \cdot BM25(t_3(q), t_3(\text{tab context}))$$

What Has Been Done?

- **Web tables [Chen et al., SIGIR 2021 ; Zhang & Balog, WWW 2018]**
  - Web tables tend to have rich context (page title, caption, headers, surrounding text, etc.)
  - Convert table into a document with multiple fields, run BM25 on each, then combine

- **Dataset discovery [Zhang et al., SIGMOD 2026; Balaka et al., SIGMOD 2025]**
  - Convert table into a single-field document, then run BM25. The document is either:
    - Concatenation of metadata (table name, column names, etc.)
    - A description generated by an LLM

This Project: Keyword Search to Find Tables

- **Background**
  - KWS over documents
  - KWS over tables
- **Goals of this project** ⟵
- **Collection of datasets**
  - The most comprehensive collection today
- **Tuning KWS systems**
- **Applying KWS systems to tables enriched with new metadata**

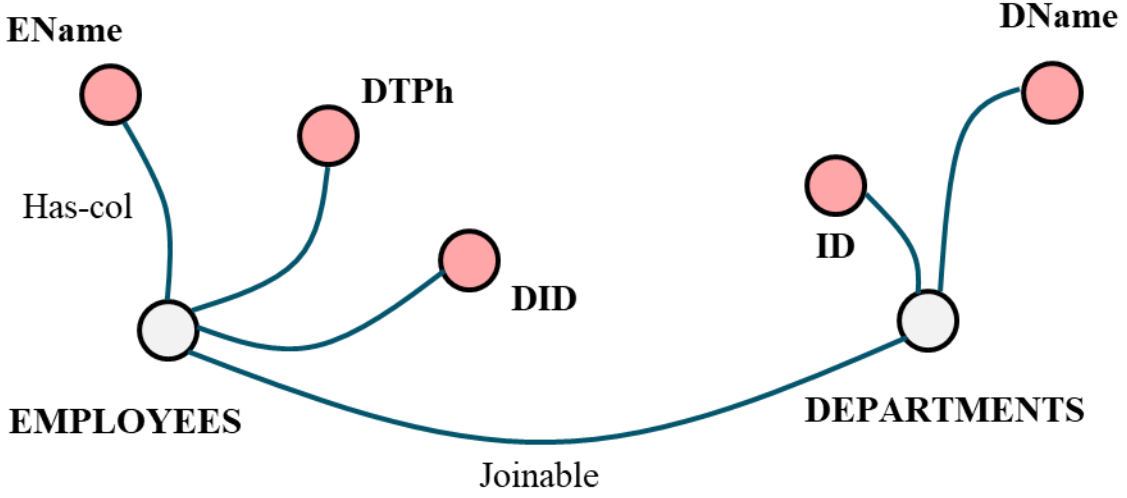
Goals: Providing KWS Capabilities for SmartCat

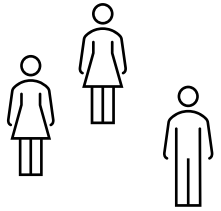
EMPLOYEES

EName	DTPh	DID
Dave Smith	4399	d_1
Jane Miller	5603	d_2
Mike Davis	2862	d_1

DEPARTMENTS

ID	DName
d_1	Sales
d_2	Legal





Keyword search
NL querying
Browsing

- **Need an out-of-the-box KWS solution for SmartCat**
- **Need to improve it over time**

Questions

Find me all tables that have employee phones
Q = “employee phone”

EMPLOYEES

EName	DTPh	DID
Dave Smith	4399	d_1
Jane Miller	5603	d_2
Mike Davis	2862	d_1

<title> EMPLOYEES</title>
<columns> Ename DTPh DID</columns>
<context>This table describes …</context>

DEPARTMENTS

ID	DName
d_1	Sales
d_2	Legal

<title> DEPARTMENTS</title>
<columns>ID DName</columns>
<context>This table was created …</context>

$$s(q, d) = w_1 \cdot BM25(t_1(q), t_1(\text{tab name})) + w_2 \cdot BM25(t_2(q), t_2(\text{col names})) + w_3 \cdot BM25(t_3(q), t_3(\text{tab context}))$$

- **Need an out-of-the-box KWS solution for SmartCat**
  - Will use BM25 TF/IDF solution
  - **How to tune tokenizers and weights?**
- **Need to improve the KWS solution over time**
  - **Can we use enriched metadata to improve KWS accuracy?**

My Contributions

- **Created the Polaris labeled dataset collection for KWS**
  - Added 3 new datasets, most comprehensive today (joint work with Ting Cai)
- **Developed a novel solution to tune parameters for KWS over tables**
- **Empirically showed that enriched metadata can significantly improve KWS**

This Project: Keyword Search to Find Tables

- **Background**
  - KWS over documents
  - KWS over tables
- **Goals of this project**
- **Collection of datasets** ⟵
  - The most comprehensive collection today
- **Tuning KWS systems**
- **Applying KWS systems to tables enriched with new metadata**

The Polaris Collection of Labeled Datasets

- **Need as many labeled datasets as possible**
  - To evaluate KWS solutions in general, to be used in my solutions in particular
- **Each labeled dataset**
  - A set of tables, a set of keyword queries, a set of relevance judgements (0/1 or 0-3)

Table r: 23_birds_1.csv [id, survey_id, site_id, species_id, obs_type, ...]

Query q: "birdwatching”

Relevance judgement (q, r) = 1 (relevant)

The Polaris Collection of Labeled Datasets

- **6 datasets, 3 new contributions (AW, EDI250, EDI20)**

Dataset	# Tables	# Queries	Min relevant #table/query	Max relevant #table/query	Avg relevant #table/query
AW	96	15	1	8	4.0
EDI250	251	20	3	43	18.5
EDI20	2,015	15	1	135	39.8
ECIR	2,100	12	94	664	450.8
WikiTables	3,361	57	1	59	14.9
WTR	4,634	60	5	110	37.4

This Project: Keyword Search to Find Tables

- **Background**
  - KWS over documents
  - KWS over tables
- **Goals of this project**
- **Collection of datasets**
  - The most comprehensive collection today
- **Tuning KWS systems** ⟵
- **Applying KWS systems to tables enriched with new metadata**

Tuning Tokenizers and Weights

d_r

Table Metadata	
Table Name	HODM_WSPD_MONTHLY
Column Names	METLOCID, DATE_TIME, AVG_WSPD, ...
Table Context	HODM – Wind Speed (Monthly) Monthly measurement …

t_1 w_1

t_2 w_2

t_3 w_3

?

$$s(q, d_r) = w_1 \cdot \mathrm{BM25}(t_1(q), t_1(\text{tab name})) + w_2 \cdot \mathrm{BM25}(t_2(q), t_2(\text{col names})) + w_3 \cdot \mathrm{BM25}(t_3(q), t_3(\text{tab context}))$$

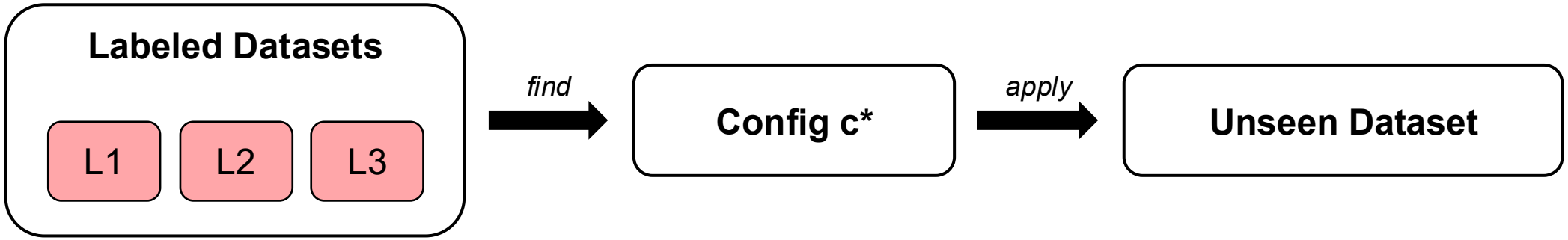
Tuning Tokenizers and Weights

- **Ideally, we have queries with known relevant tables**
  - Optimize parameters for a retrieval metric [Taylor et al., CIKM 2006]
    - Grid search over configs to maximize nDCG@10

- **Sometimes, we only have queries**
  - Rely on weak signals [Joachims, KDD 2002; Roitman, CEUR 2022]
    - Click-through rates, heuristics

- **In practice, labeled data is expensive and often unavailable** ← **Our context**
  - A new data catalog typically has no labeled data
  - No existing work on tuning tokenizers/weights for table search without labels

Our Proposed Idea



Space of Configs

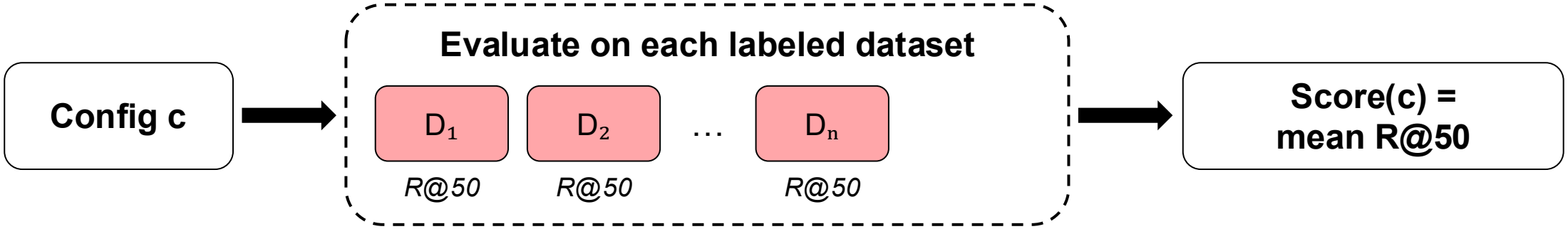
- **Recall how we score a table r**
  - $r \rightarrow d_r$ = {table_name, column_names, table_context}
  - $s(q, d_r) = \mathbf{w_1} \cdot BM25(\mathbf{t_1}(q), \mathbf{t_1}(\text{table_name}))$
    $+ \mathbf{w_2} \cdot BM25(\mathbf{t_2}(q), \mathbf{t_2}(\text{column_names}))$
    $+ \mathbf{w_3} \cdot BM25(\mathbf{t_3}(q), \mathbf{t_3}(\text{table_context}))$

- **Weights (w_1, w_2, w_3)**
  - Fix w_2 (column_names) = 1
  - w_1 and w_3 from {1/5, 1/4, 1/3, 1/2, 1, 2, 3, 4, 5}

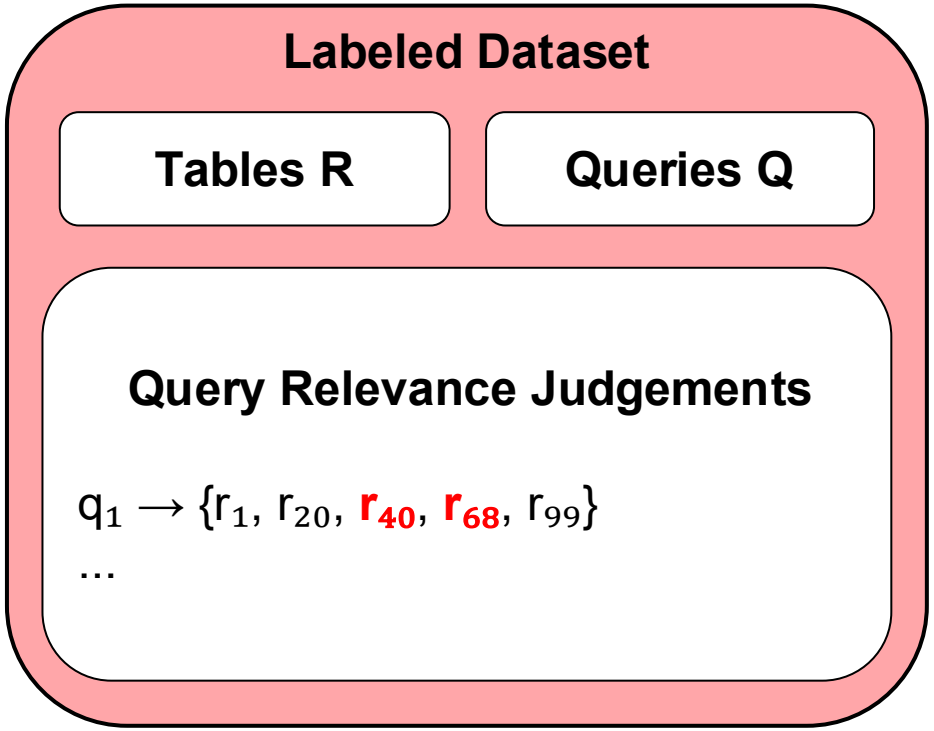
Search space =
(9 weights)^2 × (9 tokenizers)^3

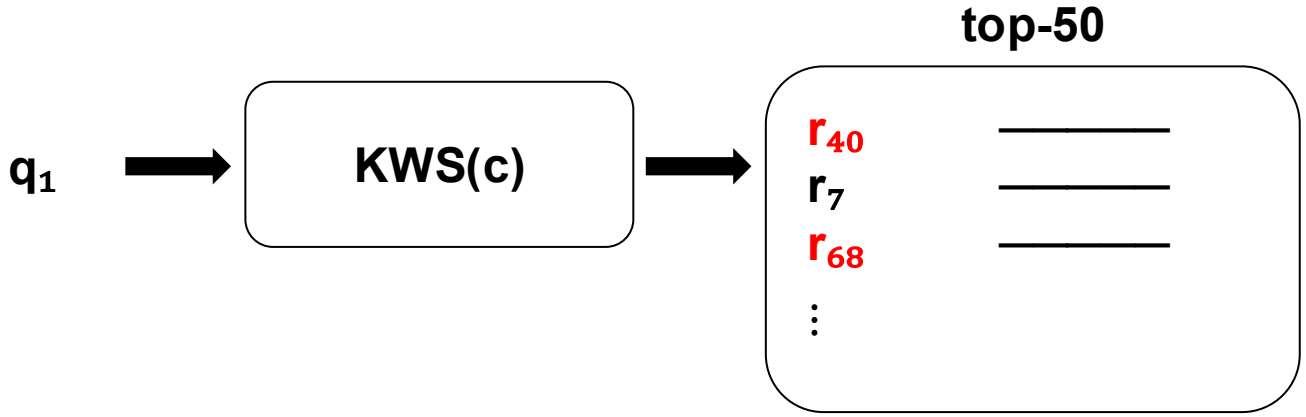
- **Tokenizers (t_1, t_2, t_3)**
  - 9 options per field:
    - Word-based (e.g., standard, whitespace, stemmed)
    - Trigram-based (character 3-grams with various split strategies)

Scoring a Config



Scoring a Config: Recall@50





For a query q:

- R@50(q, c) = |relevant ∩ top-50| / |relevant|
- R@50(q_1, c) = 2 / 5

For a dataset D:

- R@50(D, c) = 1/|Q| ∑ R@50(q_i, c)

Searching for a Good Config

- **Find the top-2 tokenizers for each field**
  - Pretend a table is a document containing *only* its table_name — try all 9 tokenizers on it
  - Rank by mean R@50 across datasets → keep the **top 2**, discard the rest
  - Repeat for column_names and table_context

- **Find the best (tokenizers × weights) config**
  - Grid search over the (remaining tokenizers × weights) space
  - Pick the config that maximizes mean R@50 across datasets

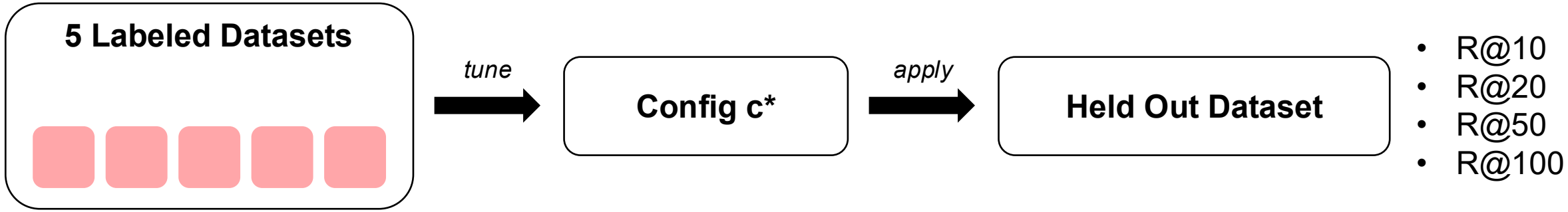
The Polaris Collection of Labeled Datasets

- **6 datasets, 3 new contributions (AW, EDI250, EDI20)**

Dataset	# Tables	# Queries	Min relevant #table/query	Max relevant #table/query	Avg relevant #table/query
AW	96	15	1	8	4.0
EDI250	251	20	3	43	18.5
EDI20	2,015	15	1	135	39.8
ECIR	2,100	12	94	664	450.8
WikiTables	3,361	57	1	59	14.9
WTR	4,634	60	5	110	37.4

Experiment

- **Cross validation**



- **Compare against baselines**
  - No published work, compare against what a practitioner would reasonably do
    - **Uniform**: Pick a reasonable analyzer per field, weight all fields equally (w = 1)
    - **Heuristic**: Pick reasonable analyzers, set weights by human judgment (e.g., table name > column names > table context)
    - **LLM**: Ask ChatGPT (GPT 5.2) for a config

- **R@10**

Method	AW	EDI250	EDI20	ECIR	WikiTables	WTR
Uniform	0.65	0.34	**0.15**	**0.02**	0.09	0.07
Heuristic	**0.68**	0.33	**0.15**	**0.02**	0.05	0.06
LLM	0.50	0.26	0.13	0.01	0.17	0.14
Our Method	0.65	**0.36**	**0.15**	**0.02**	**0.27**	**0.19**

- **R@20**

Method	AW	EDI250	EDI20	ECIR	WikiTables	WTR
Uniform	**0.79**	0.42	**0.22**	0.03	0.13	0.11
Heuristic	**0.79**	**0.43**	**0.22**	0.03	0.10	0.09
LLM	0.54	0.33	0.18	0.02	0.28	0.22
Our method	0.78	**0.43**	0.21	**0.04**	**0.39**	**0.33**

- **R@50**

Method	AW	EDI250	EDI20	ECIR	WikiTables	WTR
Uniform	**0.86**	0.62	**0.36**	0.07	0.27	0.22
Heuristic	**0.86**	**0.63**	0.35	0.06	0.21	0.17
LLM	0.54	0.49	0.25	0.05	0.49	0.39
Our Method	**0.86**	0.54	0.33	**0.08**	**0.64**	**0.61**

- **R@100**

Method	AW	EDI250	EDI20	ECIR	WikiTables	WTR
Uniform	**0.86**	0.74	**0.48**	0.10	0.65	0.61
Heuristic	**0.86**	**0.75**	0.45	0.09	0.35	0.35
LLM	0.54	0.51	0.27	0.08	0.75	0.66
Our method	**0.86**	0.73	0.44	**0.12**	**0.76**	**0.80**

Summary of Experimental Results

- **Slightly worse than best baseline on 3, significantly better on the other 3**
  - AW, EDI250, EDI20: 0-4% drop in R@100
  - ECIR, WikiTables, WTR: 1-14% gain in R@100

- **Our tuned configs generalize to unseen datasets**
  - Tuned configs perform comparable or better than baselines across all folds
  - We now have a reasonable default config for new datasets

This Project: Keyword Search to Find Tables

- **Background**
  - KWS over documents
  - KWS over tables
- **Goals of this project**
- **Collection of datasets**
  - The most comprehensive collection today
- **Tuning KWS systems**
- **Applying KWS systems to tables enriched with new metadata**

Motivation: The Vocab Gap

- **Query q = "annual precipitation"**

- **Table r = APT024 [tmpyear, janprcp, febprcp, ...]**
  - "AP" in APT024 stands for "annual precipitation"
  - "janprcp" stands for "January precipitation"

- **This table is relevant, but KWS cannot retrieve it**
  - No query term overlaps with any term in the table metadata

Why Does the Vocab Gap Problem Exist?

- **Cryptic names**
  - Table names and column names tend to be cryptic
    - APT024 [janprcp, tmpyear, …]
- **Missing descriptions**
- **Vocabulary mismatch**
  - People use different words for the same concept
    - Query q = "**revenue**"
    - Table r = quarterly_**sales**_report [**earnings**, region, ...]

Solution: Add More Metadata

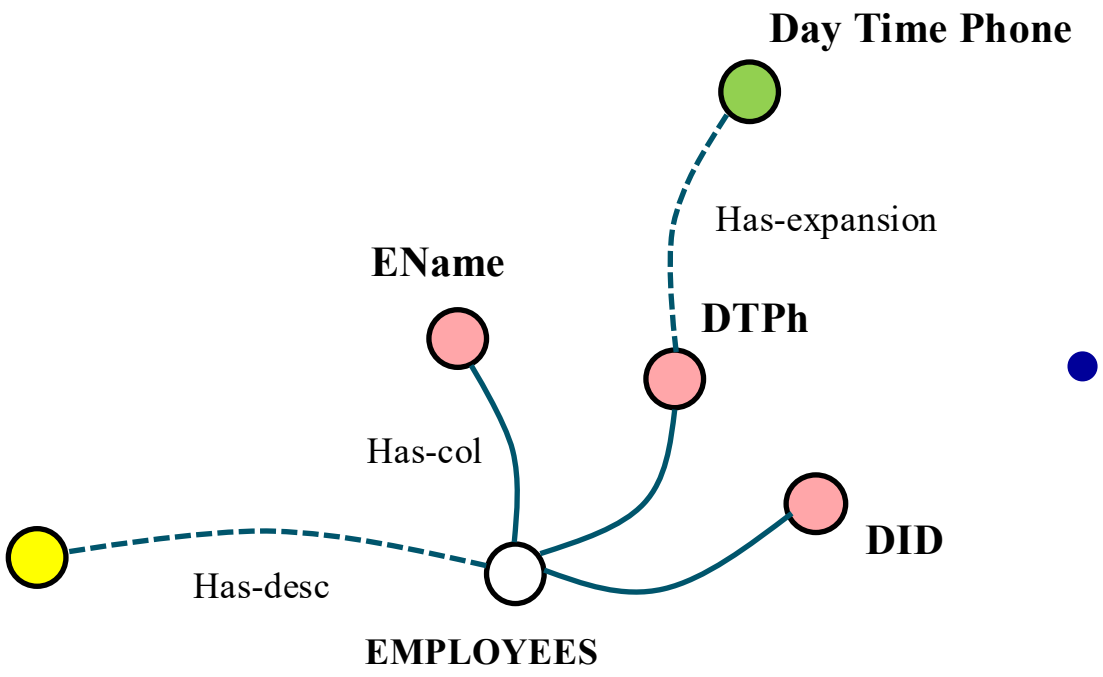
Table Column Descriptions

	sample date	lake name	ice measurement station	depth at ice measurement station	blue ice thickness	white ice thickness	depth of snow
Column Name:	sample_date	lakename	ice_sta	sta_depth	blueice	whiteice	snodep
Definition:	sample date	lake name	the station on a given lake where ice measurements were made	depth of the lake at the ice measurement station	blue ice thickness	white ice thickness	the depth of snow on top of the ice, calculated as the average of 10 snow depth samples at each sampling location.
Storage Type:	date	string	string	float	float	float	float
Measurement Type:	dateTime	nominal	nominal	ratio	ratio	ratio	ratio
Measurement Values Domain:	**Format**: YYYY-MM-DD **Precision**:	**Definition**: lake name	**Definition**: the station on a given lake where ice measurements were made	**Unit**: meter **Precision**: 1 **Type**: real	**Unit**: centimeter **Type**: real	**Unit**: centimeter **Type**: real	**Unit**: centimeter **Precision**: 0.01 **Type**: real

- **Manual addition is common**
- **Costly and labor-intensive**

Solution: Use LLMs

- **Recent work include**
  - Column name expansion [Cai et al., EMNLP 2025; Zhang et al., EMNLP 2023]
  - Table description generation [Zhang et al., SIGMOD 2026]
  - Business term association [Sheen, UW-Madison 2024]



- **Qual question: can we use enriched metadata to improve KWS accuracy?**

Representing Tables with Enriched Metadata

- **Recall: we convert each table r to a multi-field document d_r**
  - Originally, d_r had 3 fields: {table_name, column_names, table_context}

- **After enrichment, d_{r+} has 8 fields:**
  - d_{r+} = {table_name, column_names, table_context,
    table_name_expansion,
    column_name_expansion,
    table_description,
    table_tags,
    table_synonyms}

Document d_r	
Original	
Table Name	APT024
Column Names	janprcp, febprcp, tmpyear, ...
Table Context	NOAA Climate Database
Enriched (via LLM)	
Table Name Expansion	Annual Precipitation Table 024
Column Name Expansion	january precipitation, february precipitation, temporary year, ...
Table Description	Monthly and annual precipitation records from NOAA weather stations
Table Tags	precipitation, climate, weather, rainfall, NOAA
Table Synonyms	annual precipitation, yearly rainfall, rain totals

Handle cryptic names

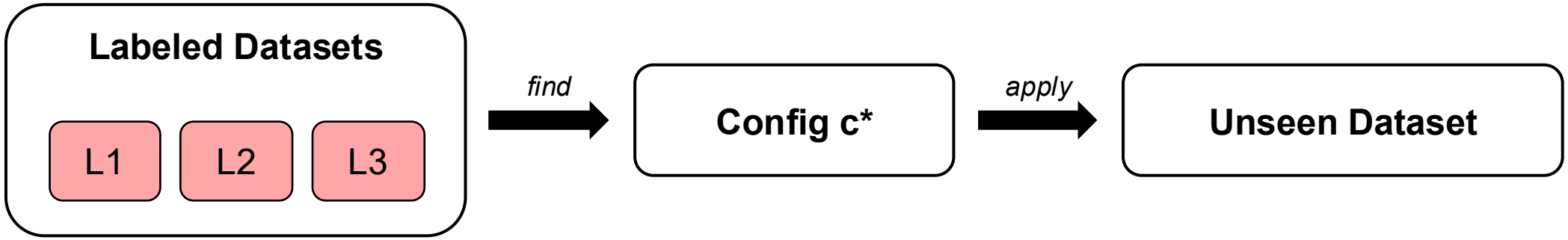
Add searchable context

Finding a Good KWS Config

- **Scoring d_{r+} against query q**

$$s(q,d) = \sum_{i=1}^{8} w_i \cdot BM25\big(t_i(q), t_i(f_i)\big)$$

- **Again, we need to find a good config for KWS over d_{r+} (with 8 fields)**
  - Weights: $w_1, \ldots, w_8$
  - Tokenizers: $t_1, \ldots, t_8$



Finding a Good KWS Config

- **Weights (w_1, w_2, …, w_8)**
  - Fix w_2 (column_names) = 1
  - Others from {1/5, 1/4, 1/3, 1/2, 1, 2, 3, 4, 5}

- **Tokenizers (t_1, t_2, …, t_8)**
  - 9 options per field:
    - Word-based (e.g., standard, whitespace, stemmed)
    - Trigram-based (character 3-grams with various split strategies)

- **Very large search space ($9^7 \times 9^8$ combinations)**

Finding a Good KWS Config

- **Step 1: Find the top-2 tokenizers for each field**
  - Pretend a table is a document containing *only* its table_name — try all 9 tokenizers on it
  - Rank by mean R@50 across datasets → keep the **top 2**, discard the rest
  - Repeat for other fields

- **Step 2: Find the best (tokenizers × weights) config**
  - ~~Grid search~~
  - For each tokenizer combination, use coordinate ascent to find its best weights
    - Optimize one weight at a time while holding the rest fixed - repeat the cycle until no improvement
    - 3 restarts to avoid local optima
  - Pick the config that maximizes mean R@50 across datasets

Experimental Results

Held-out Dataset	R@10		R@20	
	S1	S2	S1	S2
AW	0.65	**0.87**	0.78	**0.93**
EDI250	0.36	**0.48**	0.43	**0.63**
EDI20	0.15	**0.2**	0.21	**0.3**
ECIR	0.02	0.02	0.04	0.04
WikiTables	0.27	0.27	**0.39**	0.38
WTR	**0.19**	0.17	**0.33**	0.28

S1: KWS over **original** metadata (d_r - 3 fields)
S2: KWS over **original + enriched** metadata (d_{r+} - 8 fields)

Experimental Results

Held-out Dataset	R@50		R@100	
	S1	S2	S1	S2
AW	0.86	**0.96**	0.86	**0.96**
EDI250	0.54	**0.81**	0.73	**0.91**
EDI20	0.33	**0.45**	0.44	**0.59**
ECIR	0.08	0.08	0.12	0.12
WikiTables	0.64	**0.67**	0.76	**0.86**
WTR	**0.61**	0.53	**0.80**	0.77

S1: KWS over **original** metadata (d_r - 3 fields)
S2: KWS over **original + enriched** metadata (d_{r+} - 8 fields)

Discussion

- **Enriched metadata => much better on 4, same on 1, slightly worse on 1**
  - AW, EDI250, EDI20, Wikitables: 10-18% gain in R@100
  - WTR: 3% drop in R@100

- **Enriched metadata helps most when original metadata is cryptic or sparse**
  - AW, EDI250, EDI20

Which Enrichment Fields Matter Most?

- **Drop one field at a time from S2 and measure the impact**
  - (S2 – table_description) vs. S2

- **Important enrichment fields:**
  - table_description — removing hurts 3/6 datasets (1-4% drop in R@100)
  - table_synonyms — removing hurts 2/6 datasets (3-9% drop in R@100)
  - column_name_expansions — removing hurts 1/6 datasets (7% drop in R@100)

Conclusions

- **KWS over tables is a major problem for data catalogs**
  - Also a major direction in NLP research
- **This qual project**
  - Created the Polaris labeled dataset collection (most comprehensive today)
  - Developed a novel solution to tune parameters for KWS over tables
  - Empirically showed that enriched metadata can significantly improve KWS
- **Future directions**
  - Deploy our solutions in SmartCat
  - Study other scoring functions for KWS over tables: vector search, hybrid search
  - Explore other problems in data catalogs (e.g., more ways to discover tables)